Unmanned Ground Vehicle (UGV) – Part 1

IIT Hyderabad
Department of Electrical Engineering [ICDT]

EE1010
Maths For Electrical Engineering
Prof. G. V. V. Sharma

Jaideep Singh
EE26BTECH11213

Mudit Gupta
EE26BTECH11216

1. Project Overview

Project Objective:
To design, assemble, and implement a two-wheel differential drive Unmanned Ground Vehicle (**UGV**) featuring remote teleoperation via Bluetooth Classic provided by the ESP32 using the Dabble mobile interface, bidirectional DC motor actuation via an L293D quadruple half-H driver, hardware PWM speed modulation, and a decoupled DC-DC buck regulation stage.

Key Features:

1) **Dual-Wheel Differential Drive:** Actuated by two 3V–6V DC TT gear motors (1 : 48 reduction ratio) driven by an L293D quadruple half-H driver IC.
2) **Bluetooth Remote Teleoperation:** Wireless control integrated with the Dabble mobile application (GamePad module) over Bluetooth Classic on the ESP32 platform.
3) **Hardware PWM Speed Regulation:** 8-bit resolution (0–255) at a 1 kHz switching frequency, featuring directional trim calibration to eliminate motor trajectory drift.
4) **Decoupled Power Architecture:** Unregulated 8.08V motor bus paired with an adjustable LM2596 switching regulator tuned to 5.18V logic supply, eliminating brownout resets during transient motor inrush surges.

2. Components

- **ESP32-WROOM-32 Dev Module:** 1
- **L293D Quadruple Half-H Driver IC:** 1
- **DC TT Gear Motors (3V–6V, 1 : 48 Gear Ratio):** 2
- **UGV Chassis Kit (Drive Wheels, Caster Wheel, Acrylic Base Plates):** 1
- **LM2596 DC-DC Buck Converter Module:** 1
- **Rechargeable DC Battery Pack (7.40V–8.40V):** 1
- **Solderless Breadboard:** 1
- **Jumper Wires (Male-to-Male, Male-to-Female):** 25
- **Micro-USB Programming Cable:** 1

Table of Contents

1) Project Overview 1

2) Components 1

3) A Finished Look at the UGV 3

4) **Project Progress Log and Engineering Challenges** 5
- **Phase 1**: Understanding and Planning the Circuit 5
- **Phase 2**: Constructing the Initial Circuit 5
- **Phase 3**: Transitioning to Standalone Battery Powered Circuit 6
- **Phase 4**: Supply Voltage Sag and Brownout Mitigation 8
- **Phase 5**: Speed Calibration and D-Pad Expansion 11

5) Hardware Interfacing and Pin Allocation 13

6) Project Reflections and Learning Outcomes 14

7) Teamwork and Acknowledgements 15

3. A Finished Look at the UGV

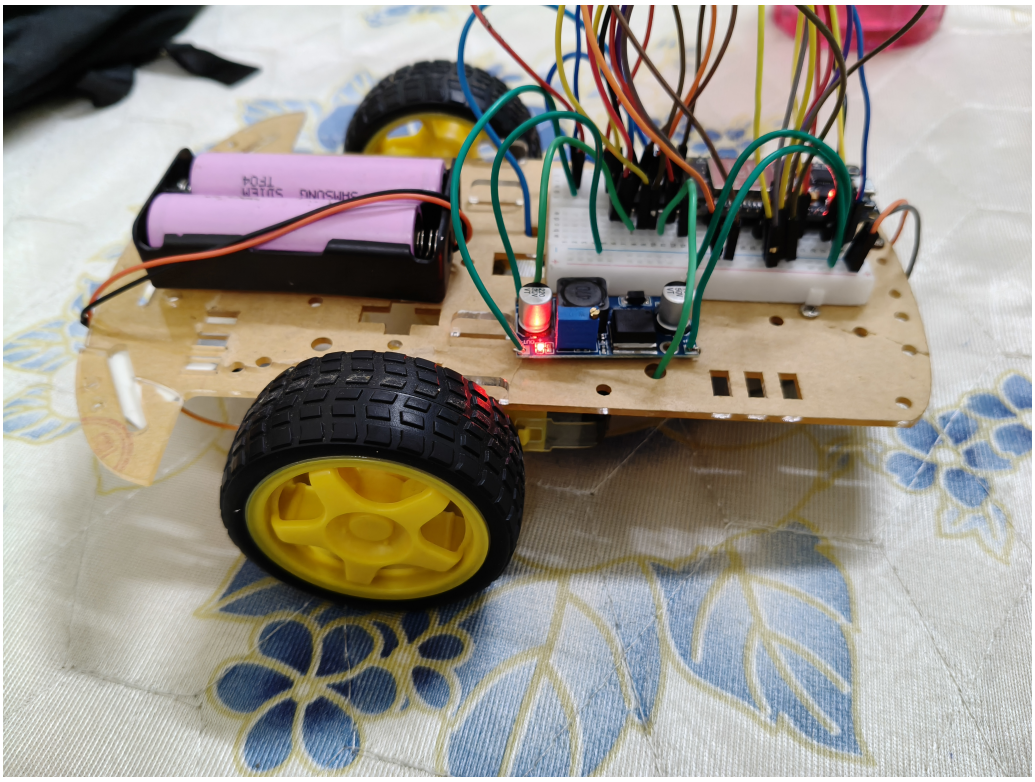


Fig. 1: Fully assembled two-wheel differential drive UGV with onboard ESP32, L293D driver, and power regulation module

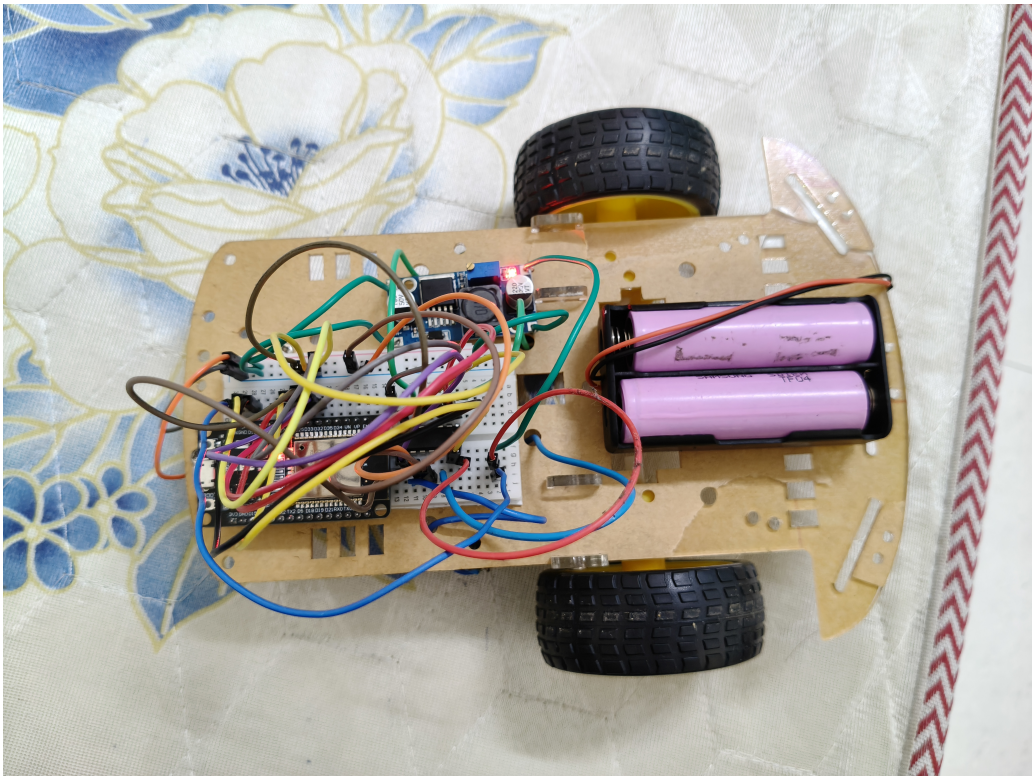


Fig. 2: Top-down layout showing motor driver routing, common ground distribution, and caster pivot alignment

System Operation and Teleoperation Architecture

The Unmanned Ground Vehicle (UGV) operates on a two-wheel differential drive mechanism steered by two 3V–6V DC TT gear motors (1 : 48 reduction ratio) paired with a free-wheeling front caster wheel. Wireless teleoperation is implemented over 2.4 GHz Bluetooth Classic, establishing an RF link between the ESP32 microcontroller and the Dabble mobile application.

The firmware imports the `<DabbleESP32.h>` header library, utilizing the **GamePad** module configured in **Digital Mode**. Incoming control packets are parsed non-blockingly using `Dabble.processInput()` inside the primary super-loop, providing real-time responsive driving without processor stall states.

Hardware PWM Throttling and Directional Control

Motor rotational speed is regulated using the ESP32's hardware LED PWM controller (`ledc`) operating at a switching frequency of 1 kHz with an 8-bit resolution (0–255 duty cycle range). These PWM signals are fed directly into the enable pins of the L293D quadruple half-H driver (Pin 1 for the Right motor and Pin 9 for the Left motor), while four dedicated GPIO lines dictate directional polarity across the internal H-bridge transistors.

Directional vectors are governed by the digital keypad (Up for forward translation, Down for reverse, and Left/Right for zero-radius axial spinning). Dynamic throttling is governed by the four right-hand digital buttons:

- **Cross (×):** Returns the robot to the **Default** cruise speed of **180** (≈ 70.6% duty cycle).
- **Square (□):** Throttles down to **Slow** speed at **130** (≈ 51.0% duty cycle) for delicate navigation.
- **Triangle (△):** Accelerates to **Fast** speed at **230** (≈ 90.2% duty cycle).
- **Circle (◯):** Engages **Maximum** speed at **255** (100% full continuous DC rail).

4. Project Progress Log and Engineering Challenges

***Phase 1**: Understanding and Planning the Circuit*

- **Objective:** Understand the mechanical and electrical architecture of the two-wheel differential drive UGV, interface the L293D dual H-bridge motor driver with the ESP32 microcontroller, and plan a dual-rail power distribution system to isolate digital logic from inductive motor noise.
- **Driver Interfacing and Logic Levels:** Studied the mechanical assembly and driver interfacing guidelines from the reference repository. The L293D quadruple half-H driver features two separate power inputs: V_{CC1} for internal logic circuitry (4.5V–7.0V) and V_{CC2} for the high-current motor stage (4.5V–36.0V). Although the ESP32 operates at 3.3V logic levels, the L293D has an input high voltage threshold (V_{IH}) of 2.3V, allowing direct GPIO drive without requiring external logic level shifters.
- **Power Distribution and Ground Plane Strategy:** Preliminary analysis showed that DC gear motors draw significant inrush currents during startup and directional shifts. If shared across an unbuffered rail, these inductive spikes cause transient voltage drops that reset the microcontroller. To prevent interference, we planned two isolated power lines on the breadboard: an unregulated bus dedicated to the motor stage (V_{CC2}) and a regulated, low-noise bus for the ESP32 and logic stages (V_{CC1}). A single common ground bus tied all negative terminals together to maintain a uniform reference potential across the entire system.
- **Assembly and Testing Roadmap:** To isolate faults systematically, we avoided assembling a fully untethered build right away. We established a phased testing approach to verify code execution, Bluetooth packet reception, and basic motor switching over a tethered USB connection first, before migrating the hardware to a standalone battery pack:

Working through USB initially $\longrightarrow$ Moving the circuit to independently work from the battery

***Phase 2**: Constructing the Initial Circuit*

- **Objective:** To verify the operational integrity of all allotted components, confirm proper signal switching across the L293D motor driver, and validate basic motor rotation on bench power prior to assembling an autonomous power system.
- **Initial Connections:** The hardware was assembled modularly on a solderless breadboard. Female-to-male jumper wires were routed from the ESP32 development board header pins down to the breadboard rows, while male-to-male jumpers established connections across the breadboard tie-points to the L293D driver IC. The two DC TT gear motors were connected to the driver output channels, and a common ground reference was established between the microcontroller ground and the driver ground pins. Broken solder on one of the motors was repaired and a fresh wire was attached. Voltage across both the cells was verified. The VIN pin of the ESP 32 used to give power for logic supply to the L293d, giving a voltage of 5.28V when connected to the laptop/mobile through the micro-usb.
- **Sketch and Verification:** Libraries in the Arduino IDE were downloaded to support the ESP32 Dev Module and Dabble utility. A basic diagnostic sketch was uploaded to the ESP32 over a tethered USB cable to validate pin responses. With the robot chassis elevated off the bench surface to prevent runaway movement, the test routine toggled the direction and enable pins to confirm that forward, reverse, and stop commands operated each motor correctly.
- **Dabble Teleoperation and D-Pad Verification:** Following low-level verification, the firmware was linked to the Dabble library to test wireless teleoperation over Bluetooth Classic. The system was temporarily powered directly from a mobile device via USB to evaluate untethered control logic before integrating the battery pack. All directional inputs from the digital GamePad D-pad (Up, Down, Left, Right) triggered the expected drive states reliably. However, physical drive tests revealed slight discrepancies between the speeds of the two DC TT motors, causing the vehicle to gently veer rather than follow an ideal straight line.



Fig. 3: Initial circuit assembly and functional verification powered via tethered USB cable

Phase 3*: Transitioning to Standalone Battery Powered Circuit*

- **Objective:** To achieve standalone, untethered operation by transitioning from continuous USB power to an onboard rechargeable DC battery supply, while establishing a safe and stable voltage level for the L293D logic and ESP32 microcontroller.

Challenge 2: Voltage Regulation for L293D Logic and Power Architecture Selection:

- **Operating Voltage Constraints:** For safe and reliable operation, the L293D motor driver requires its logic supply pin (V_{CC1}, Pin 16) to remain strictly within the range of 4.5V to 7.0V. Because the onboard battery pack delivers ≈ 8.08V–8.16V, feeding raw battery power directly into the logic circuitry would exceed absolute maximum ratings and destroy the IC. A dedicated stepping and power distribution scheme had to be decided upon.

- **Decision Making and Planning:** We evaluated three possible approaches to power the logic stage safely:
  1) **Method 1 (Linear Voltage Regulator):** Connect the battery pack to an LM7805 regulator to produce a steady 5V output. This 5V line can be supplied directly to the ESP32 via its `VIN` pin (which feeds its onboard 3.3V LDO) and routed to Pin 16 (V_{CC1}) of the L293D for logic bias.
  2) **Method 2 (DC-DC Buck Converter):** Integrate an adjustable step-down buck converter (such as the LM2596) to tune and step down the voltage with high power efficiency.
  3) **Method 3 (External Secondary Power Supply):** Use an isolated secondary source, such as a separate power bank or an auxiliary multi-cell battery holder, dedicated exclusively to powering the microcontroller.
- **Design Selection:** To minimize chassis weight, conserve limited platform space, and maintain a clean, compact layout, Method 3 was discarded. Because an LM7805 linear regulator was readily available in the hostel, it was chosen as the immediate solution to begin transitioning the vehicle to standalone operation.
- **Connection Mapping:** The LM7805 was placed across the top rows of the breadboard. The battery positive terminal was wired to the regulator input pin, the common ground was tied to the center pin, and the regulated 5V output pin was distributed to the ESP32 `VIN` header and Pin 16 of the L293D.
- **Breadboard Modification and Direct ESP32 Mounting:** In the initial tethered setup, the ESP32 was connected via female-to-male jumper wires running to the breadboard. To minimize circuit volume and prevent loose, tangled, or easily broken jumper connections while the vehicle is in motion, we decided to mount the ESP32 directly onto the breadboard. However, the ESP32 development board has a wide form factor; when pressed across the central trough, it covered the tie-points on both sides, leaving no accessible pin rows for jumper wire insertion. To solve this, one of the detachable lateral power rails (+ and GND) was unclipped and removed from the breadboard. This freed up sufficient physical clearance, allowing the ESP32 to be seated firmly while leaving an entire column of breadboard tie-points exposed for clean, direct signal routing.

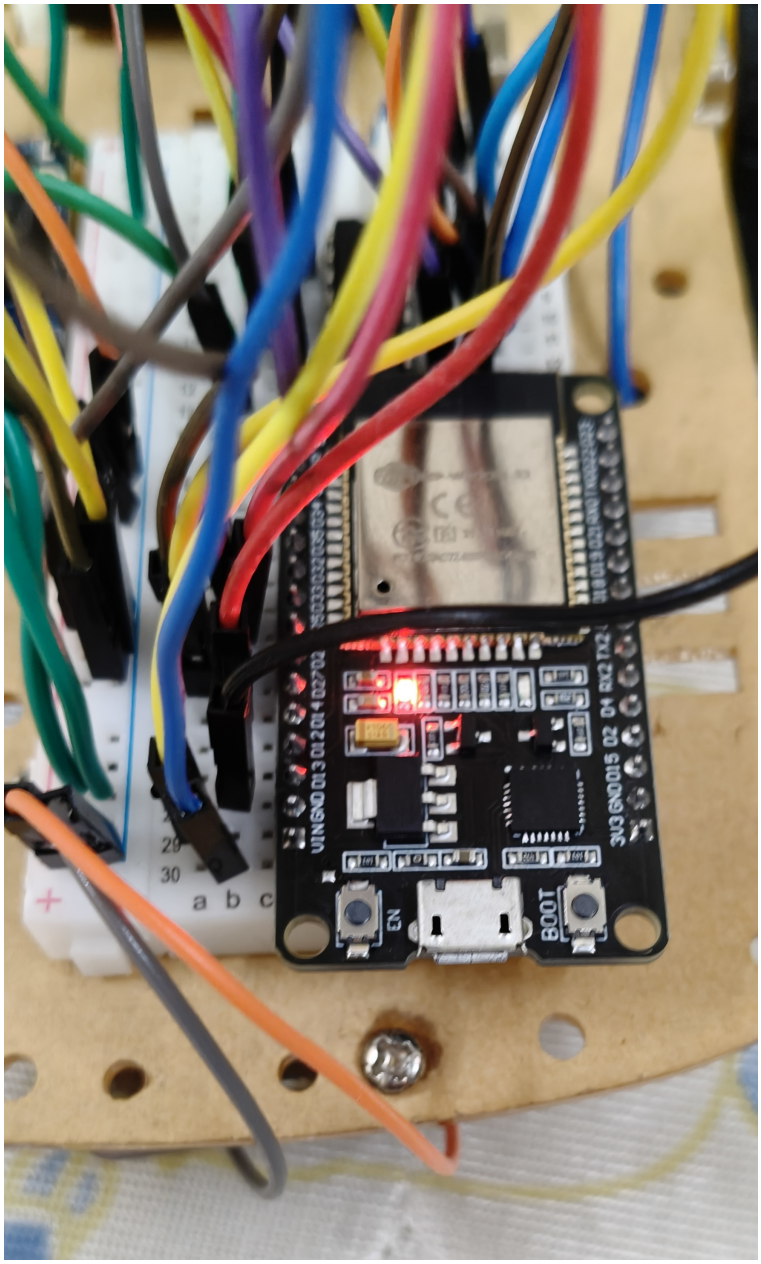


Fig. 4: Direct mounting of the ESP32 onto the breadboard following the detachment of the lateral power rail

- **Voltage Verification and Final Hardware Integration:** The regulated output across the LM7805 was measured using a digital multimeter under no-load conditions, yielding a steady 5.13V and confirming proper voltage regulation. The output was then routed to the ESP32 `VIN` pin and Pin 16 (V_{CC1}) of the L293D for logic bias, while the raw 8.08V battery rail was connected to Pin 8 (V_{CC2}) for the motor driver stage.
- **Firmware Update and Untethered Validation:** A new sketch matching the finalized breadboard pin assignments was uploaded to the ESP32. The micro-USB programming cable was then detached, and standalone untethered operation was evaluated on battery power alone.

Challenge 4: Supply Voltage Sag and Microcontroller Brownout Mitigation:

- **Observed Issue:** When running untethered on battery power, the vehicle exhibited sluggish throttle response, high-speed stalling, immediate Bluetooth disconnection in the Dabble app, and spontaneous microcontroller reboots during motor direction reversals or rapid acceleration. Serial diagnostics con-

firmed that the ESP32's internal hardware brownout detector was being repeatedly tripped (`Brownout detector was triggered`).

Phase 4A: The Software-Only Approach (Register Masking): Before altering the hardware circuit or introducing decoupling capacitors to the breadboard, an attempt was made to resolve the resets entirely at the firmware level by suppressing the brownout detector interrupt.

1) **Implementation:** The internal Real-Time Clock (RTC) controller register controlling brownout detection was cleared during `setup()` prior to peripheral initialization:

Listing 1: Suppressing Hardware Brownout Detection via RTC Register

```
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"

void setup() {
  // Suppress internal hardware brownout detection
  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);

  // Proceed with system initialization...
}
```

- **Observed Behavior & Failure Analysis:** While this routine successfully prevented the ESP32 from logging an explicit brownout abort message, it did not resolve physical system collapse:
  - *RF Radio Collapse:* Disabling the software detector cannot prevent physical voltage collapse. When the 5V rail sagged below $\approx$ 4.5V, the onboard 3.3V Low-Dropout (LDO) regulator dropped out of regulation. The 2.4 GHz Bluetooth radio suffered instantaneous power loss, causing Dabble to disconnect immediately whenever the motors engaged.
  - *Processor Lockups and Core Panics:* Under heavy motor loads, the supply dropped below the minimum operating threshold of the internal flash memory and CPU core ($\approx$ 2.3V–2.5V). Rather than executing a clean reboot, the processor encountered corrupted instruction fetches, silent hangs, and unhandled CPU exceptions .
  - *Engineering Conclusion:* Suppressing the brownout detector merely silences the system alarm; it cannot compensate for a physical deficiency of charge on the power rail. Hardware-level stabilization was strictly mandatory.

Phase 4B: Passive Buffering vs. Fundamental Linear Bottlenecks: Prior to redesigning the regulator stage, the feasibility of passive buffering on the existing LM7805 circuit was evaluated:

1) **Decoupling Limitations:** Calculations for bulk electrolytic decoupling capacitors (100 μF to 470 μF) across the breadboard rails indicated that while a capacitor could absorb high-frequency commutation noise (sub-millisecond spikes), it could not sustain the rail during multi-millisecond motor stall spikes (1.0A–1.5A). The stored energy:

$$E = \frac{1}{2} C \left(V_{\text{nominal}}^2 - V_{\text{min}}^2 \right) \tag{1}$$

   was depleted within a fraction of a millisecond under continuous motor load.
2) **The LM7805 Dropout Barrier ($V_{\text{drop}} \geq 2.0$V):** The primary failure point remained the LM7805's internal pass-transistor architecture. Maintaining a regulated 5.0V output required $V_{\text{IN}} \geq 7.0$V at all times. Because motor inrush currents through the battery's internal source resistance pulled the 8.08V pack below 7.0V during turns, the LM7805 inevitably dropped out of regulation, regardless of output capacitance.
3) **Thermal Throttling:** At an ESP32 peak RF draw of $\approx$ 240mA, linear power dissipation:

$$P_D = (V_{\text{IN}} - V_{\text{OUT}}) \cdot I_L \approx (8.08 - 5.00) \times 0.24 \approx 0.74\text{W} \tag{2}$$

caused rapid thermal rise in the un-heatsinked TO-220 package, forcing internal thermal protection to choke current delivery to the logic rail.

Phase 4C: Hardware Migration to Switching Buck Regulation: The linear architecture was fully decommissioned in favor of an adjustable LM2596 DC-DC step-down switching converter operating at 150 kHz with > 85% conversion efficiency.

1) **Bench Potentiometer Calibration:** To prevent accidental overvoltage to the microcontroller, the LM2596 was calibrated entirely unloaded. With the ESP32 and L293D removed, the 8.08V battery was connected across `IN+` and `IN-`. A digital multimeter was attached to `OUT+` and `OUT-`, and the 25-turn precision trimming potentiometer was dialed counter-clockwise through its initial dead band until the output stabilized at exactly 5.18V. This provided clean headroom above the ESP32's onboard LDO dropout point (4.5V) while remaining safely below the L293D's maximum logic rating (7.0V).
2) **Physical Node Separation (Dual-Rail Strategy):** To eliminate inductive back-EMF coupling into the digital lines, the power bus was physically bifurcated using independent 5-hole breadboard terminal rows:
   - **8.08V Motor Rail (Unregulated):** Dedicated strictly to LM2596 `IN+` and L293D Pin 8 (V_{CC2}).
   - **5.18V Logic Rail (Regulated):** Sourced from LM2596 `OUT+` strictly to ESP32 `VIN` and L293D Pin 16 (V_{CC1}).
   - **Common Star-Ground:** Battery negative, LM2596 ground terminals (`IN-`, `OUT-`), ESP32 `GND`, and L293D heat sink ground pins (Pins 4, 5, 12, 13) were bonded directly to the common ground rail to guarantee uniform reference potentials.

Phase 4D: Final Two-Tier Solution: Stable, brownout-free operation was ultimately achieved through a combined hardware-and-software architecture:

- **Hardware Tier:** The LM2596 buck converter buffered the logic rail against primary battery sag, maintaining an uninterrupted 5.18V feed to the ESP32 onboard LDO.
- **Software Tier:** Retaining `WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);` in firmware served as a secondary defense, preventing ultra-short sub-millisecond commutation noise from triggering false-positive CPU aborts during full-throttle direction reversals.

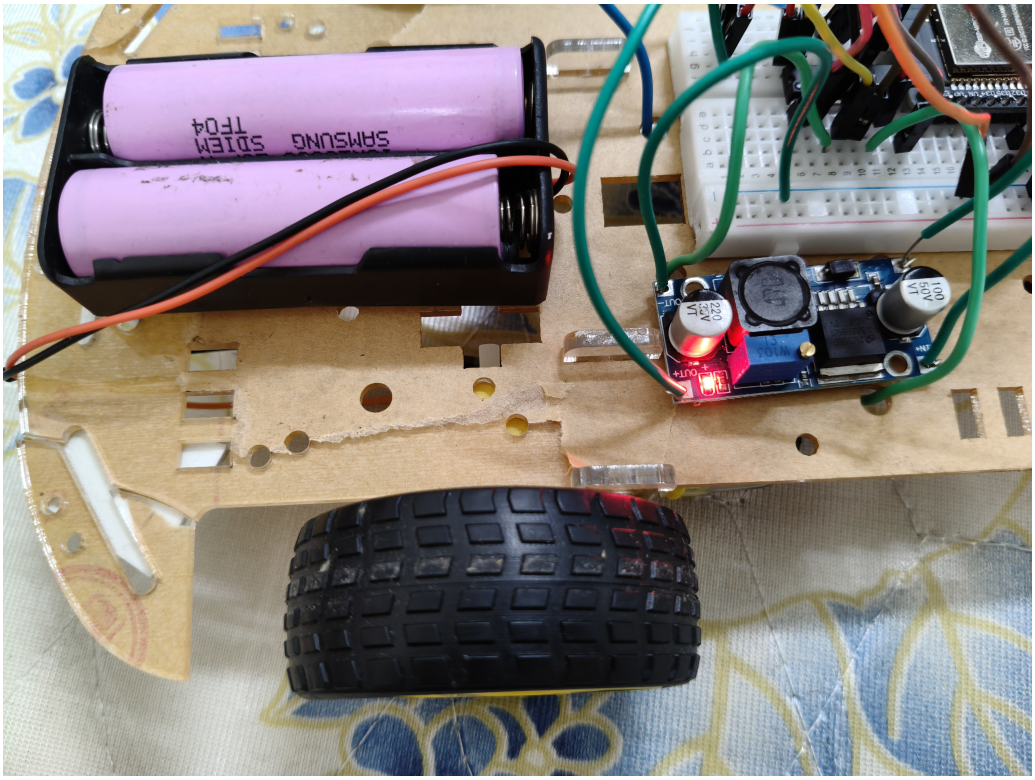


Fig. 5: Final two-tier power distribution architecture featuring the bench-calibrated LM2596 buck regulator

***Phase 5**: Speed Calibration and D-Pad Expansion*

- **Objective:** Implement dynamic multi-tier speed throttling, program complete directional motion vectors via the Dabble digital GamePad, and eliminate directional tracking drift caused by physical motor speed discrepancies.
- **Empirical Speed Calibration (Trim Multiplier):** Low-cost DC TT gear motors inherently exhibit slight mechanical and electrical tolerances in internal winding resistance, gearbox friction, and back-EMF constants. During straight-line verification runs, the left motor ran noticeably faster than the right motor, causing the UGV to veer right under identical PWM duty cycles. To ensure true linear trajectory tracking, an empirical calibration multiplier was integrated into the drive firmware. Through systematic trial-and-error speed tests across open floor runs, a trim factor of **0.96** for the left motor was determined to produce optimal balanced velocity:

$$\text{PWM}_{\text{Left}} = \text{int}\left(\text{PWM}_{\text{nominal}} \times 0.96\right) \tag{3}$$

$$\text{PWM}_{\text{Right}} = \text{PWM}_{\text{nominal}} \tag{4}$$

  This software scaling factor successfully synchronized both wheel velocities, allowing the chassis to track straight across all throttle profiles.
- **D-Pad Motion Vector Implementation:** The digital directional pad on the Dabble GamePad was mapped to govern the full differential drive state machine:
  - **Forward (UP):** Both H-bridges energized forward with the left channel scaled by the 0.96 trim factor.
  - **Reverse (DOWN):** Both H-bridges energized in reverse with balanced trim scaling.
  - **Spin Left (LEFT):** Right motor driven forward and left motor driven in reverse at symmetric unscaled speeds, executing a zero-radius counter-clockwise pivot on axis.
  - **Spin Right (RIGHT):** Left motor driven forward and right motor driven in reverse at symmetric unscaled speeds, executing a zero-radius clockwise pivot on axis.

  - **Active Dynamic Braking (IDLE):** When all directional buttons are released, all directional inputs (IN1–IN4) are grounded (LOW) and PWM duty cycles are forced to zero, actively braking the motors and preventing inertia rollout.
- **Dynamic Multi-Tier Throttle Expansion:** To provide adaptable handling across varying driving environments, the firmware was expanded with four discrete speed profiles mapped to the GamePad action buttons:
  - **Cross (×):** Cruise speed (**180**/255, ≈ 70.6% duty cycle) — set as the default initialization speed.
  - **Square (□):** Slow crawl mode (**130**/255, ≈ 51.0% duty cycle) — designed for tight, high-precision maneuvers.
  - **Triangle (△):** Fast dash mode (**230**/255, ≈ 90.2% duty cycle) — optimized for rapid straight-line travel.
  - **Circle (◯):** Maximum throttle (**255**/255, 100% full continuous DC rail) — uncapped direct battery output.

5. Hardware Interfacing and Pin Allocation

The UGV system utilizes six digital GPIOs from the ESP32: two PWM-enabled pins connected to the L293D enable lines for speed control, and four digital outputs to drive the dual H-bridge direction states. Table I details the complete physical wiring layout, electrical ratings, and signal routing across all subsystems.

TABLE I: Complete System Pin Mapping and Electrical Specifications

Subsystem	ESP32 GPIO	L293D Pin	Signal Type	Function / Electrical Specification
Right Motor Control	GPIO 33	Pin 1 (ENA)	PWM Output	Right Motor Enable (1 kHz, 8-bit, 0–255)
	GPIO 13	Pin 2 (IN1)	Digital Out	Right Motor Direction Input 1 (3.3V Logic)
	GPIO 27	Pin 7 (IN2)	Digital Out	Right Motor Direction Input 2 (3.3V Logic)
Left Motor Control	GPIO 25	Pin 9 (ENB)	PWM Output	Left Motor Enable (1 kHz, 8-bit, 0–255)
	GPIO 26	Pin 10 (IN3)	Digital Out	Left Motor Direction Input 3 (3.3V Logic)
	GPIO 32	Pin 15 (IN4)	Digital Out	Left Motor Direction Input 4 (3.3V Logic)
Motor Drive Terminals	—	Pins 3, 6	Analog Drive	Right Motor Terminals ($V_{\text{motor}} \approx$ 6.1V)
	—	Pins 11, 14	Analog Drive	Left Motor Terminals ($V_{\text{motor}} \approx 6.1$V)
Power Rails & Ground	VIN	Pin 16 (V_{CC1})	Regulated Input	Logic Bias: Regulated 5.18V from LM2596
	—	Pin 8 (V_{CC2})	Unregulated Input	Motor Output Stage: Direct 8.08V Battery Rail
	GND	Pins 4, 5, 12, 13	Ground Bus	Common Star-Ground Reference (0V)

6. Project Reflections and Learning Outcomes

A. A Beginner's Perspective on Embedded Systems

Entering the domain of embedded robotics and microcontroller programming was entirely new territory for us. Transitioning from standard software code to hands-on embedded C++ running on the ESP32 demanded an acute understanding of how software commands directly manifest as physical electromagnetic and mechanical phenomena. We quickly learned that microcontrollers cannot simply be treated as abstract logic processors; they operate in an imperfect analog environment governed by inductive kickbacks, switching transients, and strict input threshold requirements. Configuring hardware LED PWM timers and integrating asynchronous Bluetooth polling routines allowed us to realize how every clock cycle and register write directly dictates the physical behavior of the vehicle.

B. Collaborative Troubleshooting and Conceptual Growth

Our troubleshooting sessions taught us that system-level bugs are an inevitable reality of embedded design. When the vehicle initially suffered from immediate Bluetooth disconnections and brownout reboots under load, our initial instinct was to search for a quick software workaround by masking internal RTC registers. Encountering processor core freezes and RF transmitter collapse demonstrated a critical engineering principle: software instructions cannot compensate for a physical deficiency of charge on a power line. We had to step back, calculate thermal dissipation limits, analyze dropout voltage barriers, and restructure the physical power architecture using an LM2596 switching regulator. Documenting each stage in LaTeX reinforced this systematic approach, enabling us to trace hardware faults, verify signal integrity, and present our technical findings rigorously.

C. Hardware Frustrations and Physical Challenges

Despite the satisfaction of seeing our code function, the physical assembly repeatedly challenged our patience. First and foremost was the physical breadboard layout. The ESP32 development board is wide enough to bridge an entire central trough, initially blocking access to surrounding tie-points and complicating signal routing. Physically modifying the breadboard by unclipping a lateral rail gave us the necessary clearance to seat the microcontroller firmly, eliminating loose, dangling jumper connections. Furthermore, real-world mechanical tolerances proved to be a persistent hurdle. Even though both DC TT gear motors received identical PWM duty cycles, physical variations in internal gearbox friction and winding resistance forced the robot off a straight trajectory. Calibrating an empirical trim factor of 0.96 through repetitive floor trials emphasized that embedded robotics demands continuous software compensation for imperfect mechanical hardware.

D. Final Thoughts and Viva Preparation

Looking back as we prepare for our project viva, it is immensely rewarding to see how a collection of raw components evolved into a fully functional, untethered ground vehicle. What began as basic circuit diagrams and separate software routines grew into an integrated system combining hardware PWM regulation, dual-rail power isolation, Bluetooth teleoperation, and dynamic throttle control. We have developed a deep, practical understanding of H-bridge switching, buck regulation efficiency, and communication pipelines. All the frustration spent tracing brownouts, trimming potentiometers, and uncoupling breadboard rails dissipated the moment the chassis smoothly navigated the floor under complete wireless control.

7. Teamwork and Acknowledgements

Reflections on Teamwork

The successful implementation of this Unmanned Ground Vehicle was the direct outcome of focused teamwork, shared responsibility, and constant technical communication. Bridging physical mechanical assembly, breadboard electronics, and low-level firmware required a clear division of tasks. Our workflow was distributed to capitalize on individual strengths, covering everything from bench voltage calibration and power rail isolation to writing the Dabble Bluetooth interface routines and tuning motor PWM registers. When encountering complex system failures—such as brownout resets during motor commutation or trajectory drift during floor trials—we held collective brainstorming sessions to test hypotheses methodically. This collaborative dynamic significantly shortened debugging cycles and deepened our joint understanding of embedded systems design.

Acknowledgements

As a two-person team, we wish to express our sincere gratitude to everyone who contributed to the realization of this project:

- **Project Advisors & Instructors:** For outlining the clear engineering objectives of the UGV Part 1 assignment, providing foundational technical guidance, and encouraging independent circuit problem-solving.
- **Lab Staff & Teaching Assistants:** For facilitating access to the electrical laboratory facilities, diagnostic tools, digital multimeters, and chassis hardware kits.
- **Artificial Intelligence Tools:** For serving as an efficient thought partner and documentation accelerator, assisting in rapid syntax verification, register exploration, and LaTeX document structuring.
- **Peers & Classmates:** For sharing valuable feedback during untethered floor driving sessions and assisting in real-time velocity observations during speed trim calibration.